

P R O C E S S O S E L E T I V O

Desafio Técnico – Desenvolvedor de Inovação Pleno

GeoTrack

Sistema de Rastreamento de Frota com Dados Geográficos

1. O Desafio

O Instituto SENAI de Inovação em Sistemas Embarcados trabalha com projetos de natureza diversa: desde integração de dispositivos embarcados com plataformas em nuvem até sistemas de monitoramento, análise de dados e automação. O desenvolvedor de inovação pleno que buscamos precisa ter autonomia para entender um problema, tomar decisões técnicas e entregar uma solução funcional e bem documentada.

Você receberá um contexto de problema e um conjunto de requisitos. As decisões de modelagem, arquitetura, organização do código e priorização são do candidato. Esperamos que trate este desafio como trataria uma demanda real de trabalho: pesquisando o que não sabe, tomando decisões embasadas e documentando o raciocínio por trás delas.

O desafio consiste em construir uma API Django para gerenciamento e rastreamento de uma frota de carros distribuídos pelo estado de Santa Catarina. O sistema deve simular automaticamente o movimento dos veículos em background, consultar condições climáticas da região de cada carro por meio de uma API pública, e expor endpoints REST para gestão e consulta geográfica da frota. Tudo deve rodar de forma integrada via Docker Compose.

2. O Que o Sistema Deve Fazer

2.1 Gestão de Ativos

O sistema deve permitir cadastrar, listar, editar e remover carros da frota. Cada carro deve ter ao menos uma posição geográfica associada. A modelagem das entidades deve seguir a decisão do candidato. Documente as escolhas no README.

O banco deve ser inicializado com 20 carros já distribuídos em cidades de Santa Catarina. Deve-se usar cidades reais com coordenadas geográficas reais como ponto de partida.

2.2 Simulação Automática — Processamento em Background

Um processo rodando em background deve, periodicamente, atualizar a posição de cada carro (simulando movimento) e sortear aleatoriamente quais carros estão ou não funcionando. O intervalo entre cada atualização deve ser configurável via variável de ambiente.

Cada execução deve gerar logs, tanto em caso de sucesso quanto em caso de erro.
Deve ser possível acompanhar o que aconteceu em cada ciclo consultando os logs do container.

2.3 Integração com API Climática

A cada ciclo de atualização, o sistema deve consultar a previsão do tempo para a região de cada carro usando a Open-Meteo API (gratuita, sem autenticação). Armazenar apenas a última previsão consultada por carro.

A integração deve ser resiliente: se a Open-Meteo estiver indisponível, o sistema deve continuar funcionando normalmente. Implemente tratamento de exceção e circuit breaker para proteger o sistema contra falhas externas. Registrar nos logs qualquer falha na integração.

```
Open-Meteo API (gratuita, sem cadastro, sem API key):  
https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}  
    &current=temperature_2m,relative_humidity_2m,weather_code&timezone=auto  
  
Documentação: https://open-meteo.com/en/docs
```

2.4 Consulta Geográfica com PostGIS

O banco de dados deve usar PostgreSQL com a extensão PostGIS para armazenar as posições dos carros como pontos geográficos. O sistema deve expor um endpoint para consultar quais carros estão dentro de um raio (em km) a partir de uma coordenada informada.

2.5 API REST e Swagger

A API deve ter endpoints para:

- Cadastrar, listar, editar e remover carros.
- Listar carros filtrando por status (funcionando / com problema).
- Consultar carros próximos a uma coordenada e raio informados.

O Swagger deve estar disponível em /api/docs/ com exemplos de payload que permitam testar todos os endpoints sem ferramenta externa. Se não houver frontend, o Swagger é o principal meio de demonstração — deve estar completo e funcional.

2.6 Infraestrutura

Todo o sistema deve rodar com Docker Compose. Um único comando deve ser suficiente para levantar todos os serviços. Instruções claras no README.

3. Diferenciais Não Obrigatórios

Implementar apenas o que conseguir entregar com qualidade. O que ficar de fora deve ser documentado no README com justificativa.

Diferencial	Descrição
Dashboard web	Tela inicial com totais de carros ativos, carros com problema desde a última atualização, e outros indicadores relevantes.
Mapa com Leaflet	Tela com mapa centrado em Santa Catarina mostrando a posição atual de cada carro. Usar Leaflet.js. Diferenciar visualmente carros funcionando de carros com problema.
CRUD no frontend	Interface para listar, cadastrar, editar e remover carros sem precisar do Swagger.

Testes automatizados

Testes para a consulta geográfica e para o comportamento quando a Open-Meteo está indisponível.

4. Entregáveis

- Repositorio Git público (GitHub, GitLab ou similar) com histórico de commits. Não enviar .zip.
- README com: como rodar, variáveis de ambiente, o que foi implementado e o que ficou fora com justificativa.
- docker-compose.yml funcional com um único comando para levantar toda a infraestrutura necessária.
- Swagger em /api/docs/ com exemplos funcionais.
- Arquivo .env com todas as variáveis de ambiente.

5. Apresentação

Após a avaliação do entregável, será feita uma entrevista, ao qual o candidato deve apresentar a aplicação desenvolvida, com base no cumprimento dos requisitos, escolha tecnológica e arquitetura do projeto. Deve-se realizar uma demonstração prática. Ou seja, com o sistema funcionando. Não será aceito somente slides explicando o funcionamento. O candidato terá 10 minutos para poder apresentar o sistema.

OBS: Não é necessário, cumprir todos os requisitos ou todos critérios de avaliação, caso algo não seja cumprido, o item terá a nota descontada, mas não desclassifica o candidato.

Quaisquer dúvida sobre algum requisito entrar em contato por email:

rick.souza@sc.senai.br

santiago.piovesan@sc.senai.br

marcelo.nunes@sc.senai.br